Kalinga: Technical and Judge Q&A Guide

Prepared for the full-stack and AI/ML presenter | October 11, 2026

Use this as a speaking guide, not a promise that every hospital requirement is implemented. Answers describe the checked-in app and recorded demo evidence. Deployment configuration and expiring credentials should be rechecked before presenting.

1. Quick revision sheet

30-second opening: Kalinga helps patients describe their needs by voice, text, or quick buttons and connects each need to the appropriate hospital team. Clinical safety rules run first. For other requests, Nova 2 Lite and GPT-OSS 120B provide independently checked routing suggestions. Staff acknowledge and complete tasks, and patients hear the actual updates. Optional WebRTC check-ins support isolated patients.

Remember	Correct answer
Track	Ops Unleashed - Inpatient Services
Frontend / backend	React / Node.js 24 with custom HTTP/JSON handlers
Hosting / shared database	Vercel / Neon PostgreSQL
Local database	Node's built-in SQLite
Primary routing model	Amazon Nova 2 Lite through AWS Bedrock
Independent reviewer	GPT-OSS 120B through Groq
Recorded voice input	Whisper Large V3 through Groq
Spoken output	ElevenLabs Multilingual V2
Video technology	Native WebRTC; HTTP polling for signaling
Pain sequence	Notify Nursing first, then optionally collect 1-10 score
Fine-tuned model	Teammate's multilingual SetFit responder classifier
Latest recorded regression suite	253 passed; this is not a clinical accuracy percentage

Key statements to keep precise: Say GPT-OSS 120B, not 124B. Say Groq, not xAI Grok. Describe the two-model system as an application-level ensemble. Do not claim HIPAA compliance, clinical validation, interpreter replacement, or measured PPE savings.

Reading order: System and backend (2), request flow (3), models (4), SetFit (5), safety and pain (6), voice (7), video (8), security (9), evidence and cost (10), deployment (11), impact and roadmap (12), scenarios and sources (13).

2. System architecture and backend

Q1. What problem did you solve?

Patients may combine clinical and routine needs in one call. Kalinga separates those needs, routes work to the appropriate teams, and makes the handoff visible. We aim to reduce avoidable nursing interruptions and improve communication, especially for patients with language or reading barriers and those in isolation.

Q2. What is the overall architecture?

React browser interfaces call authenticated Node.js APIs on Vercel. The backend checks safety, calls model providers when appropriate, and writes tasks and notifications to Neon PostgreSQL. Staff dashboards poll for updates. Voice uses Groq and ElevenLabs; optional calls use WebRTC with database-backed signaling.

Request path: Patient UI -> authenticated API -> clinical rules or guarded model ensemble -> database transaction -> department dashboards -> real staff updates -> patient text/audio.

Voice path: Short recording -> authenticated recording API -> Groq Whisper -> original-language transcript -> normal request flow. Spoken update -> server-grounded text -> ElevenLabs -> browser playback.

Call path: Nurse invitation -> database -> patient acceptance -> SDP/ICE signaling over APIs -> WebRTC media between the participants, directly or through TURN when configured.

Q3. What backend framework do you use?

We use lightweight custom Node.js HTTP/JSON handlers. `lib/handler.cjs` contains application routes. `server.cjs` starts the local server; `api/index.js` adapts the same application to Vercel. The frontend uses React. A shared handler keeps local and hosted behavior consistent.

Q4. Which backend libraries matter?

The AWS Bedrock Runtime SDK invokes Nova. `pg` provides PostgreSQL access and pooling. Node's `crypto` handles password/session security, and `node:sqlite` supports local storage. Native `fetch` calls Groq and ElevenLabs. The Smithy/AWS signing dependencies support the optional AWS transcription adapter.

Q5. Why Vercel and Neon?

They let us deploy an HTTPS demo with persistent shared state quickly. Serverless instance memory is temporary, so accounts, requests, notifications and call signaling live in PostgreSQL. Vercel hosts the application; AWS provides Bedrock inference. We are not claiming the entire deployment runs inside AWS.

Q6. What is stored in the database?

Users, staffing states and competencies, hashed sessions, tasks with linked requests and pain details, notifications, audit events, model-review records, and video call/signaling records. Recorded request audio and call recordings are not persisted by the app. Provider-side data handling must be reviewed separately.

Q7. How do you handle simultaneous updates?

Task creation and linked requests use transactions. PostgreSQL uses an advisory transaction lock to serialize mutations across instances. Request identifiers prevent duplicate dispatch on retries. Pain updates use a revision check. This favors demo consistency; the shared lock is a scaling constraint to revisit.

3. Request flow, staff roles and notifications

Q8. Walk through one request end to end.

The session supplies patient identity and room. The API validates input, checks clinical rules and applicable model results, then saves the primary request, linked tasks and notifications together. Staff see their scoped queues, acknowledge work and complete it. The patient receives the corresponding stored status updates.

Q9. Can one request reach several teams?

Yes. The system extracts multiple intents and groups them by department. Food and water can share a Food Services task. Medication concerns can create a Nursing task alongside that service task. Linked tasks retain separate statuses, so completing a meal does not complete the Nursing request.

Q10. How are water, blankets and bathroom help routed?

Our prototype policy sends water/food to Food Services, blankets/pillows to Environmental Services, and routine toileting assistance to Bedside Care. Clinical symptoms, falls or mobility restrictions can hold mobility work for Nursing assessment. These are demo policy choices requiring validation with the intended unit.

Q11. Can the app determine whether a patient may receive water?

It cannot read diet or fluid orders from an EHR. The Food Services task tells staff to verify those orders before delivery. Kalinga identifies and coordinates a request; it does not authorize food, fluids, medication administration, or movement against clinical restrictions.

Q12. Does a role title automatically allow someone to accept a task?

No. The backend also checks configured competencies, on-shift state and action permissions. Managers and observers have different capabilities from direct responders. These are demonstration staffing controls; the app has no live hospital credential, assignment or staffing feed.

Q13. How do notifications work?

Eligible staff and the originating patient receive database-backed notifications. Dashboards poll approximately every two seconds. Optional browser alerts require permission and the open application. Closed-browser push, SMS, email and integration with hospital paging equipment are not implemented.

Q14. What if nobody is available, or a request is overdue?

Uncovered queues alert available oversight accounts. Overdue Nursing escalation is evaluated during polling and is deduplicated. It is not a continuously running hospital paging worker. Staff response times and patient-to-provider assignments still need validation and integration with real unit workflows.

4. Models, ensemble decisions and failures

Q15. Which models are active, and why?

Nova 2 Lite through Bedrock is the primary structured intent extractor. GPT-OSS 120B through Groq supplies an independent second opinion. Whisper Large V3 handles recorded multilingual speech, and ElevenLabs Multilingual V2 handles spoken output. Each model has a separate job; they are not all called for every request.

Q16. What are the exact model identifiers?

The configured identifiers are `us.amazon.nova-2-lite-v1:0`, `openai/gpt-oss-120b`, `whisper-large-v3`, and `eleven_multilingual_v2`. Bedrock has been verified in `us-west-2`. The application sends provider requests using server-side credentials; an enabled setting alone is not evidence of a successful inference call.

Q17. Is this a mixture of experts you trained?

Our routing orchestration is an application-level ensemble of independently called models. We did not train a new combined mixture-of-experts network. Our teammate separately fine-tuned a SetFit responder classifier. Its training source is available on the `fine-tuned-model` branch.

Q18. How do you validate model output?

We check response structure, allowed intents/departments, literal evidence from the patient's message, clinical precedence and routing policy. Invalid or low-confidence responses receive conservative handling. The current confidence cutoff is a guard, not calibrated clinical probability. The server derives linked tasks rather than accepting arbitrary model-created assignments.

Q19. What counts as agreement?

We compare guarded department sets, intent kinds and primary priority. We do not compare prose or average confidence values. Both experts must produce valid outputs before we record agreement. Their agreement can expose consistency, but two models can share an error; it does not establish clinical correctness.

Q20. What happens when models disagree?

A validated clinical concern takes precedence; among conflicting clinical outputs, the more urgent priority wins. Conflicting nonclinical routes require Nursing review. Explicit operational tasks may remain separate, while mobility can be held for assessment. Known clinical triggers already bypass this comparison.

Q21. What happens when a model times out or hits a rate limit?

Calls have bounded waits and run concurrently. If one expert validates, its guarded route can be used and the other failure is recorded. If neither validates, deterministic handling remains. Clear routine requests may use rules; unclear input requires Nursing review. Groq 429 responses trigger a warm-instance cooldown.

Q22. How do you address hallucinations and instruction-like input?

Model output is treated as untrusted data. Evidence must match the request, categories are bounded, and server policy controls dispatch. Receipts do not invent staff acknowledgement or arrival times. These safeguards reduce some failure modes; adversarial, ambiguous and unseen phrasing still requires testing and human oversight.

5. Teammate's fine-tuned SetFit model

Q23. What did your teammate fine-tune?

The imported training script fine-tunes `sentence-transformers/paraphrase-multilingual-MiniLM-L12-v2` using SetFit and a one-vs-rest multi-label classifier. It predicts responder categories rather than generating free-form medical advice. The published model identifier is `priyadp1/setfit-multilingual`.

Q24. What are the six labels?

Registered Nurse, Provider, Charge Nurse, Patient Care Technician, Environmental Services, and Dietary. The responder mapping sends RN/Provider/Charge votes to Nursing review, maps PCT to Bedside Care, and maps EVS/Dietary to their service queues. SetFit does not independently authorize physician dispatch or urgency.

Q25. How was it trained and evaluated?

The imported script uses a grouped train/test split, a fixed random seed, and checks for evaluation-text overlap. Training settings include batch size four, two epochs and ten iterations. Development and holdout tooling is included. We did not rerun training or verify a held-out accuracy claim during the source import.

Q26. Where is the code, and how is it attributed?

The `fine-tuned-model` branch contains `models/setfit-multilingual`: training, datasets, evaluation, routing and experiment files. The source was imported unchanged from the teammate's repository with its MIT license, author attribution, pinned revision and checksums.

Q27. What is the SetFit prediction interface?

The adapter expects six `{label, score}` entries, one for each responder category. It validates known, unique labels and finite scores between zero and one before applying the responder mapping. These bounded predictions support routing review rather than independently authorizing clinical action.

Q28. How should the fine-tuned model be evaluated?

Validate held-out multilingual and compound requests, calibrate thresholds, align label ordering, and reconcile routing policy. The teammate model routes water and blankets to PCT; Kalinga currently routes them to Food Services and EVS. Its six labels and sequence limit do not cover every app task. Preserve independent clinical guards.

6. Clinical safety and pain

Q29. What is your central safety principle?

Known clinical and emergency triggers route through server rules before model inference. Pain and medication concerns stay with Nursing. Urgent P1 conditions add a parallel Medical Team task. We call these tested prototype guardrails, not an airtight or clinically validated safety system.

Q30. Why notify Nursing before asking for a pain score?

Collecting extra information should not delay the initial notification. The server commits the Nursing request and alerts first; Kalinga then asks an optional 1-10 question. A score updates that same ticket and alerts Nursing again. It does not create an unrelated replacement request.

Q31. Is a high pain score automatically an emergency?

The score is patient-reported information, not a diagnosis or a replacement for symptom assessment. Known urgent symptoms determine escalation independently of the number. A low score does not downgrade an existing emergency. Pain reports need an appropriate clinical response from staff.

Q32. What if the patient skips or cannot answer?

The original Nursing request stays active. Patients can skip the optional score or reopen an open non-emergency pain request to update it later. Whole-number scores must be between one and ten. Dose, room number and unrelated/ambiguous answers should not be treated as pain ratings.

Q33. What if staff already acknowledged the request before the score arrives?

The pain update preserves acknowledgement, updates the existing Nursing ticket and sends a new score notification. Revision checks reject stale conflicting updates; identical retries avoid duplicate alerts. Linked service tasks remain separate and do not receive the clinical pain details.

Q34. What if pain and bathroom help occur together?

Pain reaches Nursing immediately. Mobility/toileting assistance may be held for Nursing assessment when symptoms or restrictions make delegation uncertain. A medication-only concern can coexist with routine bedside assistance, but the app should not assume every combined clinical request is safe for independent mobility dispatch.

Q35. Can this replace the call bell, medical assessment or prescribing?

No. This demo does not activate hospital paging or replace the bedside call bell. It coordinates requests and staff follow-up. Diagnosis, assessment, medication decisions and urgent response remain clinical responsibilities. Unknown symptom phrasing and transcription errors are important residual risks to evaluate before a pilot.

7. Voice, multilingual input and accessibility

Q36. How does multilingual recording work?

The patient selects a language, taps Speak, talks and taps Stop. MediaRecorder captures a short clip, and the patient-only API sends it to Groq Whisper Large V3. The transcription endpoint retains original-language text. The clip has a 30-second client limit and a one-megabyte decoded size limit. Groq transcription reference.

Q37. Is Whisper translating every request into English?

No. Our recording path uses transcription, not the audio-to-English translation endpoint. The selected language is supplied to Whisper. Routing models process multilingual text and produce structured task information. This is request interpretation and localization, not a certified translation of a medical conversation.

Q38. Which languages have you actually checked?

Hosted synthetic recordings passed in Hindi, French, Spanish and Mexican Spanish. Mexican Spanish uses `es-MX` in the app and `es` for Whisper. The UI offers additional languages, but we have not validated every one. Physical microphones, accents and mixed-language speech still need native-speaker testing.

Q39. What if speech recognition is wrong or detects silence?

The app rejects empty or low-quality segment results and shows retry/type options. Routine transcripts remain drafts for review. Explicit recognized pain can send before a score question, so erroneous pain recognition is a residual risk. Quality thresholds are heuristics; they cannot guarantee absence of hallucinated or missed words.

Q40. How does 'Hey Kalinga' work, and why can accents fail?

Hands-free mode uses browser speech recognition, separately from recorded Whisper input. It handles name/greeting variants and recognition alternatives, but depends on browser language support and speech quality. It runs only when enabled on a visible page and pauses for playback, calls or other capture. Accent accuracy is not established.

Q41. Is Amazon Transcribe being used live?

The AWS streaming adapter is prepared, but live authentication/recognition has not been verified. Bedrock bearer access does not automatically authenticate Transcribe. The current manual multilingual path uses Groq Whisper; browser recognition handles hands-free mode. We should not describe AWS Transcribe as the active verified speech service.

Q42. How do you make spoken replies gentle and truthful?

ElevenLabs reads bounded, server-grounded receipt and update templates. Pain prompts acknowledge discomfort without diagnosing or promising an arrival time. Spoken acknowledgement reflects a real staff action. Patients can stop audio, disable readback or retry blocked playback. Large text and a turn-taking chime support the interaction.

Q43. Does this replace medical interpreters?

No. Multilingual support is intended for basic needs and routine communication. Medical discussions requiring accurate interpretation should continue to use qualified medical interpreters or certified interpreters as appropriate. We have not measured interpreter-related delay reduction, and video calls do not have live translation or interpreter conferencing.

8. Video-call technology and lifecycle

Q44. What framework powers video calls?

We implement native WebRTC with `RTCPeerConnection`, `getUserMedia` and browser media tracks. React renders the call interface. The Node backend manages invitations, participant access and signaling records. We did not integrate a managed conferencing SDK; this is a two-party check-in implementation.

Q45. Explain the call flow in simple terms.

An eligible nurse acknowledges a Nursing request and offers a call. The patient accepts video or audio-only, or declines. The browsers obtain media permission and exchange session descriptions and connection candidates through authenticated APIs. WebRTC then carries the audio/video. Ending the call leaves the care request open.

Q46. What are SDP, ICE, STUN and TURN?

SDP describes the media session negotiated by offer/answer. ICE evaluates connection candidates. STUN helps discover a reachable network address. TURN relays media when direct connectivity is unavailable. Our demo uses direct connections with STUN; the app supports TURN configuration, but a live relay is not provisioned/verified here.

Q47. How can video work with serverless hosting?

Signaling uses short authenticated HTTP requests and PostgreSQL records, polled approximately every second. The nurse creates the offer and the patient responds with an answer. ICE candidates use the same channel. Vercel handles signaling APIs; the media flows through WebRTC rather than the API function.

Q48. Who can offer or join, and when does access expire?

An on-shift RN/charge nurse with Nursing competency can offer a call on an acknowledged Nursing request. Only its nurse and patient can access signaling. The patient chooses whether to join. Invitations expire after 90 seconds; accepted sessions have a 15-minute limit and one active call per participant.

Q49. Why might the iPad and phone fail to connect?

Permission denial, autoplay restrictions, missing media support and restrictive network/NAT conditions can prevent a call. Begin on the same Wi-Fi, keep both tabs open and allow camera/microphone access. Safari may require Enable call audio. Cross-network reliability needs a working TURN relay and physical-device tests.

Q50. What controls and failure cleanup are implemented?

Video or audio-only acceptance, decline, microphone mute, camera-off and end-call controls are implemented. Ending releases tracks and closes the peer connection. Connection failures leave the request active. Rejoining a negotiated session after reload is not implemented; end and reoffer. Calls are not recorded or transcribed by the app.

Q51. Is video clinically useful, and have you proven PPE savings?

We aim to support appropriate remote conversations for isolated patients and avoid some unnecessary room entries. Bedside assessment and physical assistance still require staff. Avoided visits, PPE use and feelings of isolation have not been measured; these are hypotheses for a supervised pilot, not demonstrated outcomes.

Q52. Is the video call itself translated or monitored by the AI?

No. AI request transcription and routing happen separately from the live call. The current call is direct communication between the nurse and patient, with no AI interpretation, captions, recording or medical analysis. Its clinical conversation may require a qualified interpreter and hospital-approved communications tooling.

9. Authentication, privacy and HIPAA readiness

Q53. How does authentication work?

Passwords use salted scrypt hashing. Login creates a random session token; only its SHA-256 hash is stored. Production cookies are HttpOnly, Secure and SameSite=Lax. Sessions expire after eight hours. Failed-login limits and origin checks add protections. The demo does not implement hospital SSO, MFA or self-service signup.

Q54. How do you prevent access to another patient's data?

Identity and room are derived from the session, not trusted client fields. Backend queries scope patient requests to the owner and staff requests to authorized departments or oversight roles. Mutations check permissions and eligibility. Call signaling is limited to the two participants. Frontend hiding alone is not the access control.

Q55. Do Food Services or EVS see clinical details?

Linked service tasks contain focused service summaries rather than the full clinical request. Pain details and cross-team model-review records are kept away from nonclinical department views. Request notifications are scoped. This is task-level minimization; production still requires a full data-access and information-disclosure review.

Q56. Where do audio and text leave the application?

Recorded speech goes through the backend to Groq for transcription. Applicable request text goes to Bedrock/Groq for routing, and grounded reply text goes to ElevenLabs for synthesis. Browser hands-free recognition may use the browser provider's speech service. The app does not persist raw recordings; provider retention cannot be assumed.

Q57. Are API keys exposed in the browser?

Model and synthesis credentials remain in server environment variables. The client calls authenticated app endpoints. TURN uses time-limited credentials when configured; its shared secret stays server-side. Credentials shared during development should be rotated before continued use, and expiring Bedrock access must be renewed.

Q58. Is Kalinga HIPAA compliant?

We do not claim HIPAA compliance. This is a synthetic-data hackathon prototype. Hospital adoption would require review of data flows, safeguards, applicable vendor agreements, retention, access policies and operations. HHS identifies vendor compliance and business associate agreements as relevant to covered telehealth services. HHS guidance.

Q59. Why are demo credentials unsuitable for a hospital rollout?

The app seeds demonstration accounts with a shared initial password and a small unit roster. Real deployment needs individual account lifecycle management, stronger identity controls, verified credentials/assignments and governance. Our stored sessions and permissions are implemented controls, not evidence of enterprise identity readiness or a completed security assessment.

10. Evidence, latency, cost and scale

Q60. What has actually been tested?

The latest recorded regression suite passed 253 tests. Hosted smoke checks verified PostgreSQL persistence, separate sign-ins, scoped requests, real Bedrock/Groq calls, ElevenLabs output and pain updates. Four hosted synthetic language recordings were transcribed by real Whisper. Native WebRTC was checked using generated browser streams. Physical patient use was not evaluated.

Q61. Can you claim 100 percent accuracy?

No. Passing a synthetic regression set verifies those expected cases, not real-world model or clinical accuracy. A recorded ensemble run passed 32/32 application targets, but only 7/16 model-layer cases had two validated experts; rate limits/fallbacks were distinguished. We have not proven the ensemble improves clinical outcomes or accuracy.

Q62. What latency have you measured?

A prior 32-case structured-intent benchmark reported about 2.8 seconds p95 for that run. One hosted ensemble example recorded Nova at 1345 ms and Groq at 921 ms. These are scoped observations, not an SLA. Recording duration, speech synthesis, polling, cold starts and network conditions add user-visible delay.

Q63. How would you evaluate routing properly?

Use a held-out dataset labeled with unit clinicians, covering single/multiple intents, negation, historical symptoms, ambiguity, emergencies and languages. Report urgent-miss rate, wrong/extra department rates and per-intent precision/recall. Compare rules, one model, the ensemble and SetFit separately, including latency and provider failures. Measure complete application outcomes too.

Q64. How would you evaluate voice and accessibility?

Test native speakers and real devices in realistic noise, including accents, mixed-language speech and short score answers. Measure transcription errors that change routing, task completion and need for correction. Ask patients about text size, pacing and chime usefulness. The four synthetic-language checks do not establish device or accessibility performance.

Q65. What does each request cost?

We have not measured production unit cost. Applicable requests may use two classification calls, recorded speech uses a transcription call, and readbacks use synthesis calls. Hosting, database and future TURN relay costs also matter. Clinical bypass avoids classification calls; duplicates avoid repeat dispatch. Estimate costs from actual usage before proposing savings.

Q66. Can the current architecture scale to a whole hospital?

It supports a small shared demo, not a validated hospital-scale service. The global transaction lock, complete scoped-history reads and frequent polling would need redesign and load testing. Hospital scale also needs multi-unit tenancy, roster integration, durable notification workers, monitoring and stronger availability/incident procedures. Better infrastructure alone does not establish workflow safety.

11. Installation, deployment and troubleshooting

Q67. How do I run it locally?

Install Node.js 24, clone the project, enter `hhcode`, run `npm ci`, copy `.env.example` to `.env`, and run `npm start`. Open `http://127.0.0.1:8123`. Local demo password: `kaalinga-demo`. Provider keys enable live AI/voice; local history uses SQLite.

Q68. How do I test different users simultaneously?

Run `npm run test:users` in another terminal. Use `patient.localhost:8124`, `nurse.localhost:8125` and other listed hosts. Host-only cookies separate sessions while the database is shared. Different ports on the same hostname do not isolate cookies. Hosted tests can use separate devices, profiles or private windows.

Q69. What are the numbered Vercel deployment steps?

1. Import `sairam782/HealthHack` and set the root to `hhcode`.
2. Select Node.js 24, build command `npm run build`, output `public`.
3. Connect dedicated Neon PostgreSQL and set its pooled URL as `DATABASE_URL`.
4. Set a private `DEMO_PASSWORD` and server-side provider credentials.
5. Configure the provider and ensemble flags below; deploy.
6. The first API request initializes tables and demo accounts.
7. Verify independent sign-ins, persistence, routing, pain alerts, speech and device calls.

Q70. Which environment settings matter?

Use `AI_PROVIDER=bedrock`, `AWS_REGION=us-west-2`, `BEDROCK_MODEL_ID=us.amazon.nova-2-lite-v1:0`, `AI_ROUTING_MODE=ensemble`, `AI_SECONDARY_PROVIDER=groq`, `AI_SECONDARY_MODEL=openai/gpt-oss-120b`, `VOICE_RECORDING_PROVIDER=groq`, and `VOICE_INPUT_PROVIDER=browser`. Secrets: `DATABASE_URL`, `DEMO_PASSWORD`, `AWS_BEARER_TOKEN_BEDROCK`, `GROQ_API_KEY`, `ELEVENLABS_API_KEY`. Enter actual values only in secure environment configuration.

Q71. What about AWS hosting or a failing deployment?

An AWS container path is prepared for App Runner/ECS with PostgreSQL/RDS, but it was not deployed or container-tested here. For Vercel, check database/password settings, root directory, build output and provider errors. Model failure can use fallback; database/network failure cannot guarantee request delivery. Keep the physical call bell workflow available.

Q72. How do I troubleshoot voice or video quickly?

For voice, select the correct language, use Multilingual recording, tap Stop, check microphone permission and retry blocked audio playback. For calls, acknowledge the Nursing request, offer a call, accept on the patient device, join on both sides and check media permissions. Start on the same Wi-Fi; TURN is needed for dependable restrictive-network connectivity.

12. Product impact, differentiation and next steps

Q73. What is distinctive about Kalinga?

The core is coordinated, multi-intent routing with explicit clinical precedence and task-level privacy. Voice access, genuine lifecycle updates and optional virtual check-ins support that workflow. The app aims to give nurses useful clinical requests while allowing other teams to handle suitable routine work. The concept still needs comparison with existing hospital systems.

Q74. What benefits have you demonstrated versus proposed?

We demonstrated synthetic requests reaching separate queues, persisted status updates, real model/speech calls and generated-stream video connectivity. Reduced nurse interruptions, faster real care, reduced interpreter delays, PPE savings and improved patient connection are intended benefits. A supervised pilot must measure them before we claim they occurred.

Q75. How would you pilot it?

Start with a single unit and clinician-approved task/delegation policy. Evaluate in shadow mode before dispatch influences real work, preserve existing call-bell response, and involve nurses, PCTs, support staff, interpreters and patients. Track errors, correction burden, response time, acceptance and language access. Resolve governance/security requirements before live patient data.

Q76. What is planned next?

Expand comparative evaluation of the fine-tuned model; provision/test TURN; improve device/accent checks; add hospital identity and staffing integration; and evaluate durable notifications. We also plan a physical remote with dedicated buttons for patients who prefer or need an alternative to voice and touchscreen controls. The remote is not implemented.

Q77. Does the app integrate with EHRs or hospital call-bell hardware?

Not yet. The current app has no EHR, FHIR/HL7, diet-order, room-hardware or hospital-paging connection. These are possible future integration areas, subject to hospital workflow and security review. The demonstration roster and request history are not an official RWJUH operational system.

Q78. How should I describe my contribution?

Describe only work you personally owned or reviewed: full-stack implementation, model/API integration, guarded routing, voice workflows, persistence, staff notifications, WebRTC signaling and deployment, as applicable. Attribute the fine-tuned SetFit model to your teammate. Do not imply that you personally trained all foundation models or verified every hospital requirement.

13. Difficult scenarios, demo plan and references

Scenario: 'I need water, bathroom help and medication.' Nursing handles medication, Food Services gets water, and Bedside Care can receive routine toileting help under policy. Added symptoms or restrictions can hold mobility for Nursing assessment. Each team's task has an independent status.

Scenario: 'I cannot breathe and I want water.' Known breathing distress triggers P1 Nursing plus Medical Team escalation without model latency. Service work never replaces urgent response; any separated fluid task still requires diet/fluid-order checks. Use the bedside emergency workflow; this demo does not page the hospital.

Scenario: 'My pain is two, but my chest feels tight.' Known urgent wording outranks the low score. During follow-up it raises the existing clinical request and adds the medical escalation without downgrading earlier urgency.

Scenario: 'It says received; did a nurse acknowledge?' Received means the server stored/dispatched the request. Acknowledged means authorized staff acted on it. Spoken templates preserve this difference. We never simulate acknowledgement as though a real staff member accepted the request.

Scenario: 'Can a nurse interpret a French conversation through the video feature?' The call does not provide translation. Use appropriate professional language access for medical discussion. Basic request transcription does not replace medical interpretation.

Scenario: 'Why include a fine-tuned responder model?' Our teammate's multilingual SetFit model focuses on six actionable responder categories. Its training source and published weights support comparative evaluation alongside the foundation models. Bounded label predictions remain subject to clinical rules and routing policy.

Scenario: 'All tests passed, so is the hospital safe to use it?' Tests cover specific synthetic behavior. Real patients, unknown phrasing, devices, staffing, networks and clinical protocols remain unvalidated. The next step is supervised evaluation, not a claim of clinical readiness.

Before presenting: Keep separate patient, nurse and service sessions open. Recheck provider credentials, microphone/playback permissions and network conditions. Rehearse one routine multi-team request, one pain request with score-after-notification, and nurse acknowledgement. Demonstrate video only after a successful rehearsal on the intended devices. Keep visible text as a fallback if audio fails.

A useful answer when evidence is missing: 'That is implemented/configurable/planned as follows, but we have not validated that outcome yet. Here is the check we would run next.' Choose the correct status before describing it.

Project references:

- Public repository and live application.
- Fine-tuned model branch and teammate source.
- Deployment, expert routing, patient voice, video calls, verification evidence, and staff policy.
- Recorded hosted evidence: `artifacts/expert-deployment-check.json`, `artifacts/multilingual-voice-deployment-check.json`, and `artifacts/pain-followup-deployment-check.json`. Generated-stream call evidence: `artifacts/video-call-browser-check.json`. These local artifacts are ignored by Git.

Primary external references: Groq speech-to-text, WebRTC connections, and HHS telehealth technology guidance. Provider documentation is not evidence of Kalinga's measured accuracy or compliance.